Telecomm Support Triage Report

Course Name: Agentic AI

Institution Name*:* Medicaps University – Datagami Skill Based Course

Student Name(s) & Enrolment Number(s):

Sr no	Student Name	Enrolment Number
1.	Mohd Dilshad Khan	EN22CS301602
2.	Nishit Sharma	EN22CS301658
3.	Paridhi Gupta	EN22CS301683
4.	Mohd Suhail Khan	EN22CS301604
5.	Nirmal Kant Shrivastava	EN22CS301656

Group Name: Group 08D6

Project Number: AAI-19

Industry Mentor Name:

University Mentor Name:Dr. Nishant Shrivastav

Academic Year: 2025-2026

Problem Statement & Objectives

1. Problem Statement

Telecom companies receive thousands of customer complaints daily through various channels such as chat, email, SMS, and web portals. These complaints include issues related to network failure, slow internet, recharge problems, billing errors, SIM activation, and service disruptions.

Currently, most telecom support systems rely heavily on manual ticket handling and rule-based systems, which leads to:

- Delayed response times
- Misclassification of complaints
- High workload on support agents
- Inefficient ticket prioritization
- Poor customer satisfaction

There is a need for an intelligent, automated system that can:

- Understand customer complaints in natural language
- Classify issues based on intent and urgency
- Extract important information automatically
- Retrieve accurate troubleshooting steps
- Generate professional responses
- Create and manage support tickets efficiently

Therefore, this project aims to design and develop an **AI-powered Telecom Support Triage Agent** that leverages Large Language Models (LLMs), intent classification techniques, and Retrieval-Augmented Generation (RAG) to automate complaint handling and improve the overall efficiency of telecom customer support systems.

2. Project Objectives

The primary objective of this project is to design and develop an intelligent AIbased system that automates the classification, prioritization, and management of telecom customer complaints.

- **Main Objectives**

1. **To automate complaint classification**
   Identify the intent and category of customer issues such as network problems, billing issues, recharge failures, or service disruptions.
2. **To prioritize complaints based on urgency**
   Automatically determine whether a complaint is high, medium, or low priority using AI-based intent analysis.
3. **To extract key information from customer messages**
   Use LLM-based techniques to extract important entities such as mobile numbers, dates, complaint references, and issue types.
4. **To implement Retrieval-Augmented Generation (RAG)**
   Retrieve relevant troubleshooting steps from a knowledge base using vector embeddings and semantic search.
5. **To generate automated professional responses**
   Provide customers with clear and structured replies using Large Language Models.
6. **To implement a ticket management system**
   Automatically create, store, update, and manage complaint tickets using a database (MongoDB).
7. **To reduce manual workload and improve efficiency**
   Minimize human intervention in basic support queries and improve response time.
8. **To enhance customer satisfaction**
   Provide faster, consistent, and accurate solutions to customer problems.

3. Scope of the Project

The scope of this project is focused on designing and developing an AI-based automated complaint handling system specifically for telecom customer support. The system aims to improve efficiency, reduce manual workload, and enhance customer experience through intelligent automation.

- **In-Scope Features**

1. **Natural Language Complaint Processing**
   The system will accept customer complaints in natural language format (text input).
2. **Intent Classification and Prioritization**
   It will classify complaints into categories such as network issues, billing problems, recharge failures, and service disruptions.
   It will also assign urgency levels (High, Medium, Low).
3. **Entity Extraction**
   The system will extract key information such as:
   - Mobile number
   - Date of issue
   - Complaint reference
   - Issue type
4. **Knowledge Retrieval using RAG**
   Troubleshooting steps and support policies will be stored in a knowledge base and retrieved using vector search techniques.
5. **Automated Response Generation**
   The system will generate professional and structured replies using Large Language Models (LLMs).
6. **Ticket Creation and Management**
   The system will automatically generate ticket IDs and store complaint details in a database (MongoDB).

7. **Scalable API-Based Architecture**
   The system will be designed using FastAPI and LangChain to allow future integration with web portals or messaging platforms.
8. **To enhance customer satisfaction**
   Provide faster, consistent, and accurate solutions to customer problems.

Proposed Solution

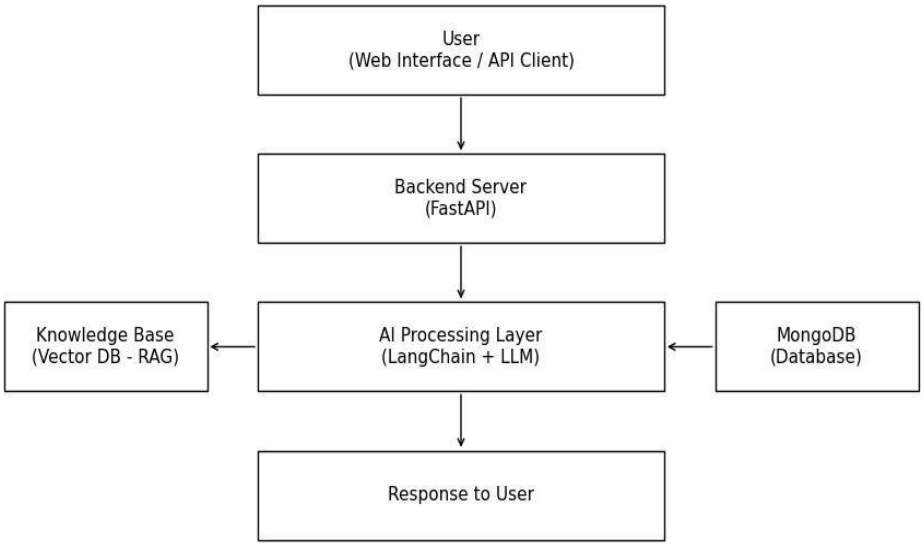
1. Key features

- Automated Complaint Classification – Automatically categorizes customer complaints into relevant issue types.
- Urgency Level Detection – Assigns priority levels (High, Medium, Low) based on complaint content.
- Entity Extraction – Extracts important details such as mobile number and issue type from text.
- Ticket ID Generation – Generates a unique ID for each complaint for tracking purposes.
- REST API-Based Backend – Provides complaint handling through structured REST APIs.
- MongoDB Integration – Stores and manages ticket data in a NoSQL database.
- Automated Response Generation – Generates structured responses based on the identified issue.
- Scalable Architecture – Designed to support future enhancements and AI integrations.
- Swagger Documentation – Provides interactive API documentation for testing and demonstration.

2. System Architecture

The AI-Powered Telecom Support Triage Agent is an intelligent system designed to automate the processing and management of telecom customer complaints. The system leverages Artificial Intelligence technologies such as Natural Language Processing (NLP), Large Language Models (LLMs), and Retrieval-Augmented Generation (RAG) to improve the efficiency and accuracy of customer support operations.

The system accepts customer complaints through a web interface or API endpoint. Once a complaint is received, it is processed through multiple AI-driven modules. First, the system performs intent classification to identify the type of issue (e.g., network problem, billing issue, recharge failure) and determines its urgency level. Next, entity extraction is carried out to identify important information such as mobile number, date of issue, and complaint details.



Process Flow

The process flow of the AI-Powered Telecom Support Triage Agent describes the step-by-step execution of the system from receiving a customer complaint to generating a response and storing a support ticket.

Step-by-Step Process Flow

Step 1: Complaint Submission

The customer submits a complaint through the web interface or API endpoint.

The complaint is provided in natural language format.

Step 2: Input Validation

The FastAPI backend receives the complaint and validates the input data to ensure it meets the required format and structure.

Step 3: Intent Classification

The system analyzes the complaint using a classification model (BERT or ZeroShot) to determine:

- The type of issue
- The urgency level (High, Medium, Low)

Step 4: Entity Extraction

The system extracts important information from the complaint, such as:

- Mobile number
- Date of issue
- Issue description
- Location

Step 5: Knowledge Retrieval (RAG)

The Retrieval-Augmented Generation (RAG) module retrieves relevant troubleshooting steps from the knowledge base stored in the vector database.

Step 6: Response Generation

The Large Language Model (LLM) generates a professional and structured response using:

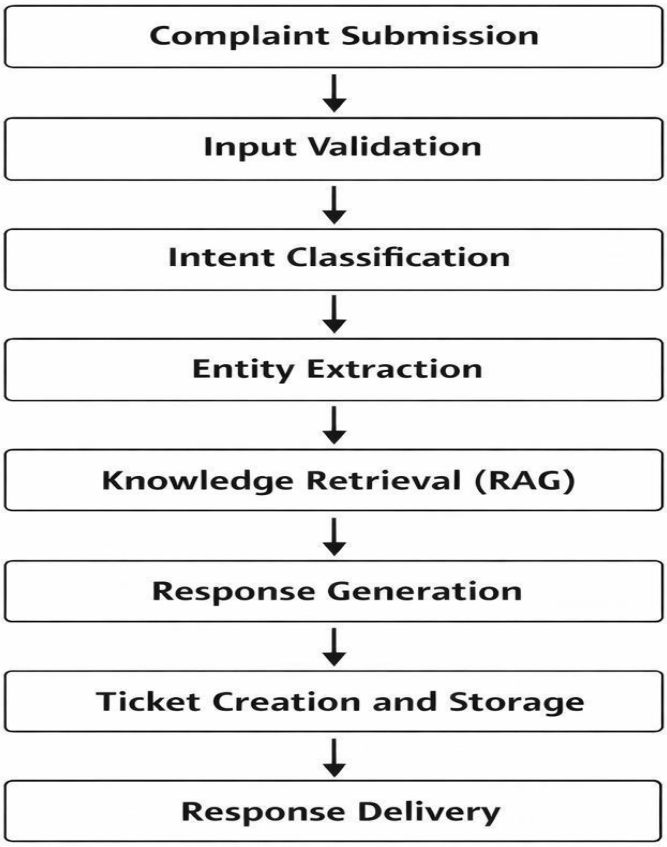
- Classified intent
- Extracted entities
- Retrieved troubleshooting information

Step 7: Ticket Creation and Storage

A unique ticket ID is generated. All complaint details and system-generated response are stored in MongoDB for tracking and status management.

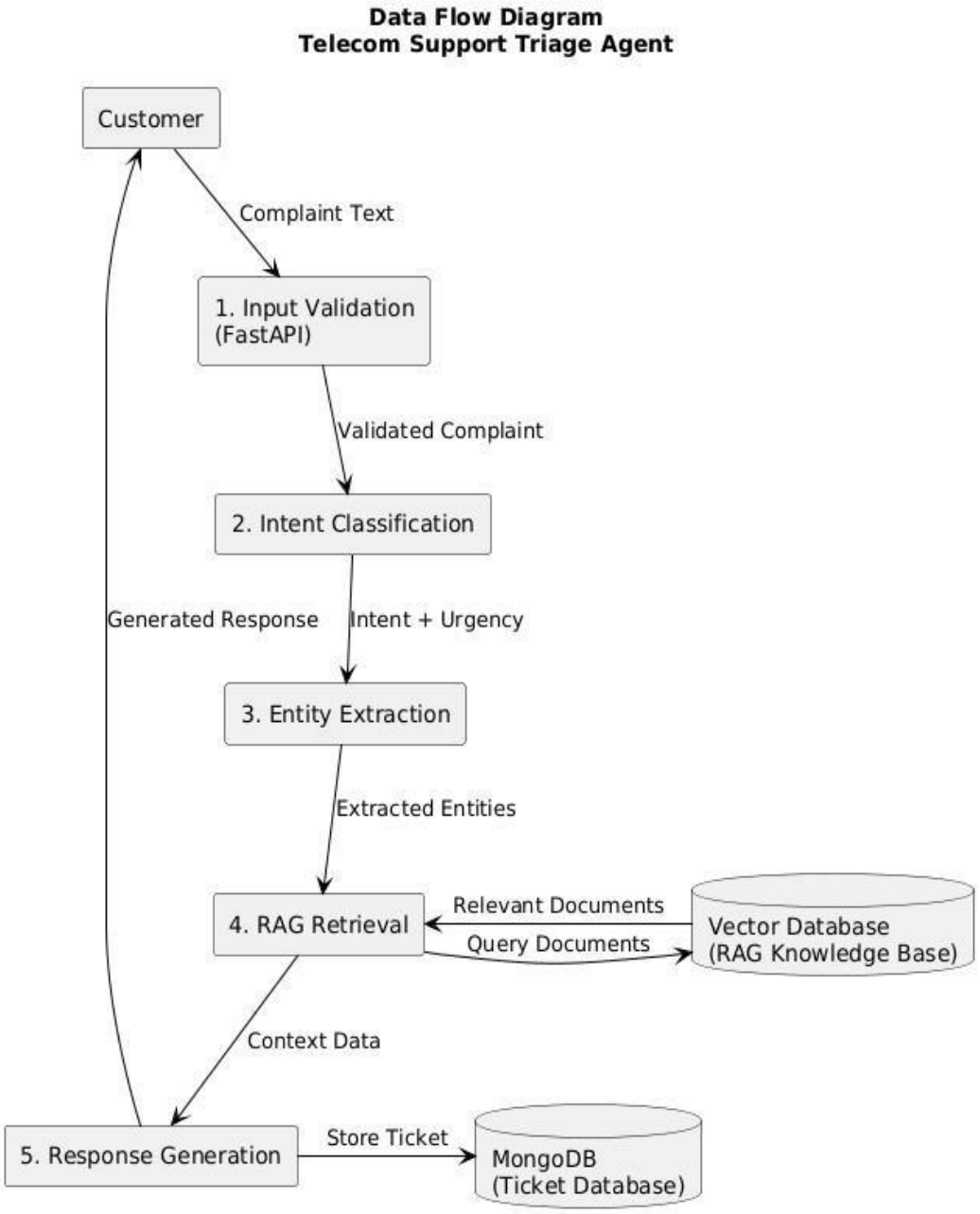
Step 8: Response Delivery

The final response is sent back to the customer via API or web interface.



Information flow

The information flow represents how information moves within the Telecom Support Triage Agent system. It illustrates the interaction between the external entity (customer), internal processing modules, and data storage components. The diagram focuses on how complaint data is received, processed, stored, and returned as a structured response.



3. Tools & Technologies Used

Programming Language

- Python-Used for backend development, AI integration, API creation, and overall system implementation.

Backend Framework

- FastAPI-Used to build RESTful APIs for handling customer complaints and managing tickets.

AI & Machine Learning Technologies

- Large Language Model (LLM API)-Used for response generation and entity extraction.
- BERT / Zero-Shot Classification-Used for intent detection and urgency classification.
- LangChain Framework-Used for workflow orchestration and integrating LLM with RAG pipeline.

Knowledge Retrieval (RAG Components)

- Vector Database (FAISS / Chroma)-Used to store document embeddings and retrieve troubleshooting information using semantic similarity search.
- Embedding Model-Converts text into vector representations for efficient retrieval.

Database

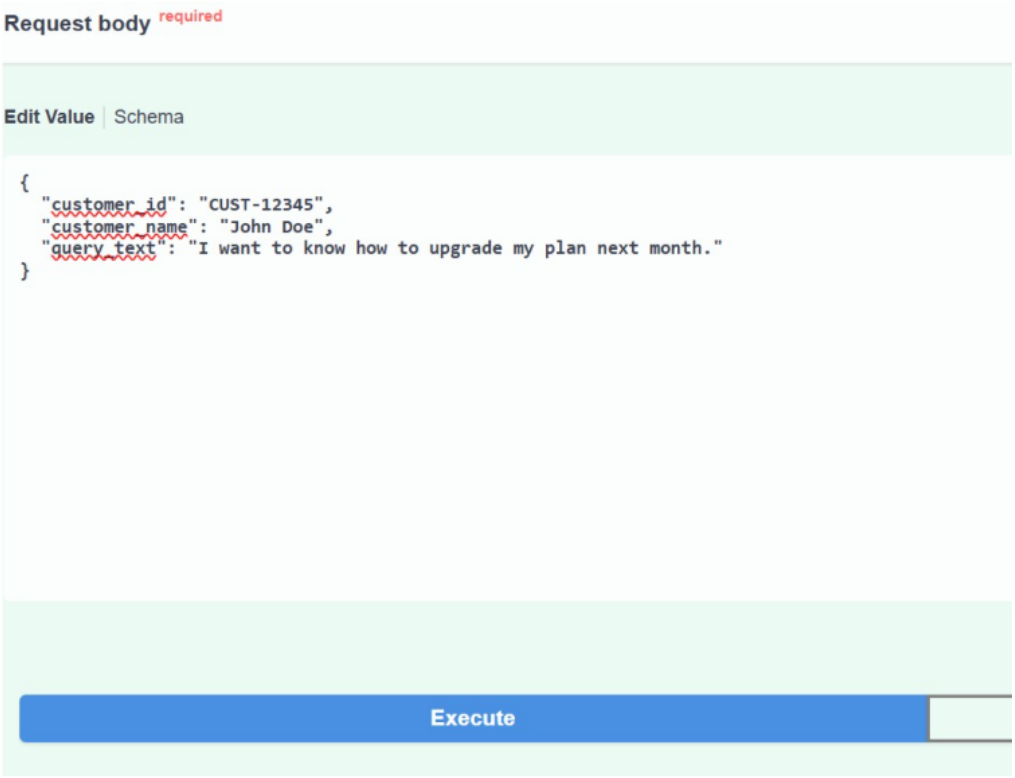
- MongoDB-Used to store complaint details, ticket IDs, status, urgency level, and timestamps.

Development Tools

- VS Code / Jupyter Notebook
- Postman (API Testing)

Results & Output

1. Screenshots / outputs



This screenshot shows the request body entered in the Swagger UI for testing the /submit_query API endpoint. The user provides structured input including customer ID, customer name, and query text. This example demonstrates a normal service-related query where the customer wants to upgrade their plan. The system accepts the JSON input and processes it for classification and ticket generation.

Curl

```
curl -X 'POST' \
  'http://localhost:8000/submit_query' \
  -H 'accept: application/json' \
  -H 'Content-Type: application/json' \
  -d '{
  "customer_id": "CUST-12345",
  "customer_name": "John Doe",
  "query_text": "I want to know how to upgrade my plan next month."
}'
```

Request URL

```
http://localhost:8000/submit_query
```

Server response

Code	Details
200	Response body

```
{
  "ticket_id": "TKT-9453BBD5",
  "customer_id": "CUST-12345",
  "query_text": "I want to know how to upgrade my plan next month.",
  "category": "device_support",
  "priority": "medium",
  "priority_score": 50,
  "automated_response": "Thank you for reaching out. We have logged your request and an agent will assist you soon.",
  "timestamp": "2026-02-24T10:18:25.913043"
}
```

Response headers

```
content-length: 343
content-type: application/json
date: Tue,24 Feb 2026 04:48:25 GMT
server: uvicorn
```

This screenshot displays the successful server response generated after executing the API request for a normal query. The system automatically generates a unique ticket ID, categorizes the query under the appropriate service category, assigns a priority level, calculates a priority score, and generates an automated response message. The timestamp indicates when the ticket was created.

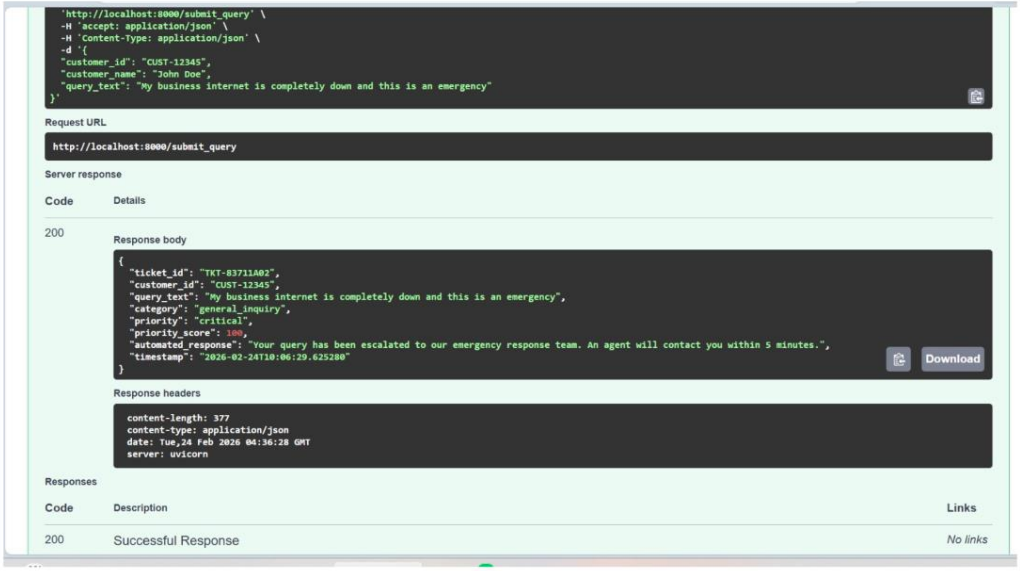
Request body required

Edit Value | Schema

```
{
  "customer_id": "CUST-12345",
  "customer_name": "John Doe",
  "query_text": "My business internet is completely down and this is an emergency"
}
```

Execute

This screenshot shows another test case where the user submits a high-priority complaint indicating that their business internet is completely down. The input JSON includes the customer details and emergency query text. This test demonstrates the system's ability to detect critical issues based on the complaint content.



This screenshot presents the server response for the emergency complaint. The system successfully identifies the issue as critical, assigns the highest priority level, generates a high priority score, and produces an automated escalation message. This confirms that the triage logic correctly differentiates between normal and urgent complaints.

Response body


```
[
  {
    "ticket_id": "TKT-83711A02",
    "customer_id": "CUST-12345",
    "query_text": "My business internet is completely down and this is an emergency",
    "category": "general_inquiry",
    "priority": "critical",
    "priority_score": 100,
    "automated_response": "Your query has been escalated to our emergency response team. An agent will contact you within 5 minutes.",
    "timestamp": "2026-02-24T10:06:29.625280"
  },
  {
    "ticket_id": "TKT-B1D3BC32",
    "customer_id": "CUST-12345",
    "query_text": "I want to know how to upgrade my plan next month",
    "category": "device_support",
    "priority": "medium",
    "priority_score": 50,
    "automated_response": "Thank you for reaching out. We have logged your request and an agent will assist you soon.",
    "timestamp": "2026-02-24T10:03:32.844283"
  },
  {
    "ticket_id": "TKT-4EB08904",
    "customer_id": "CUST-12345",
    "query_text": "I want to know how to upgrade my plan next month",
    "category": "device_support",
    "priority": "medium",
```


Download

Response headers

```
content-length: 1065
content-type: application/json
date: Tue,24 Feb 2026 04:38:26 GMT
server: uvicorn
```

This screenshot shows multiple ticket records generated and stored in the system. Each record contains the ticket ID, customer ID, query text, category, priority level, priority score, automated response, and timestamp. This demonstrates that the system maintains structured and trackable records for all submitted complaints.

2. Reports / dashboards / models

The Telecom Support Triage Agent system generates structured JSON reports for every submitted complaint. Each complaint is converted into a ticket containing detailed information such as ticket ID, customer ID, query text, detected category, priority level, priority score, automated response message, and timestamp. These structured responses serve as system-generated reports for tracking and managing customer support requests.

Although the current implementation is backend-focused and does not include a graphical dashboard interface, the stored ticket records function as structured reporting data. All complaint entries are saved in the database, enabling administrators or support agents to retrieve, monitor, and analyze complaint trends.

The system is built using defined data models that structure and validate incoming requests and outgoing responses. The primary model includes customer details and query text as input parameters, while the output model consists of categorized

and prioritized ticket information. These models ensure consistency, reliability, and scalability of the system.

In future enhancements, a web-based dashboard can be implemented to visualize ticket statistics, priority distribution, and complaint trends using charts and analytics tools.

3. Key outcomes

The Telecom Support Triage Agent successfully automates the initial complaint handling process in a telecom support environment. The system is capable of classifying customer complaints into predefined categories and determining the urgency level based on the complaint content. It effectively extracts relevant entities from unstructured text and converts them into structured data for storage and processing.

Each complaint is assigned a unique ticket ID, ensuring proper tracking and management of support requests. The system securely stores all complaint records in the MongoDB database, enabling structured data management and retrieval. The backend application operates efficiently using FastAPI and Uvicorn, demonstrating high performance and scalability.

Overall, the project reduces manual effort in the triage process, improves response time, and provides a modular architecture that can be further enhanced with advanced AI or LLM-based models in the future.

Conclusion

The Telecom Support Triage Agent project successfully demonstrates the development of a Python-based backend system designed to automate the initial complaint handling process in a telecom support environment. The system effectively classifies customer complaints, determines urgency levels, extracts relevant entities, generates structured responses, and stores ticket information in a MongoDB database.

Through this project, a modular and scalable architecture was implemented using FastAPI and Uvicorn, ensuring efficient API performance and structured data validation using Pydantic. The integration of rule-based classification and entity extraction highlights how unstructured customer inputs can be converted into meaningful, actionable data.

This project provided practical exposure to backend API development, database integration, system design concepts, and software architecture

documentation including HLD and sequence diagrams. Overall, the system reduces manual effort in complaint triage and lays a strong foundation for future enhancements such as AI-driven classification, machine learning models, or LLM integration.

Future Scope & Enhancements

The current implementation of the Telecom Support Triage Agent focuses on rulebased complaint classification and structured ticket generation. In the future, the system can be enhanced by integrating Machine Learning or Large Language Models (LLMs) to improve the accuracy of intent detection and entity extraction. AI-based models can help in handling complex and ambiguous customer complaints more effectively.

The system can also be extended by adding a web-based dashboard for support agents to monitor, update, and manage tickets in real time. Role-based authentication and authorization mechanisms can be implemented to strengthen security and access control.

Further improvements may include integration with real telecom CRM systems, SMS or email notification services for ticket updates, and deployment on cloud platforms such as AWS or Azure for better scalability and availability. Performance optimization, caching mechanisms, and automated logging and monitoring tools can also be incorporated to make the system production-ready.

Overall, the project provides a strong foundation that can be expanded into a fully intelligent and enterprise-grade telecom support automation system.